MKT 681E Final Project: Deep Reinforcement Learning for Trading

Albert Guo

December 14, 2023

Abstract

In this project, I ran empirical experiments on financial market data to study how observable state dimension, training data volume, efficient exploration, and proper reward shaping affect the performance of a model-free deep reinforcement learning (DRL) algorithm. A trading DRL algorithm is trained and tested on minute-level equity prices of `ASML` from November 25, 2016 to November 25, 2023; and is compared against the NASDAQ-100 index (`QQQ`) as well as the holding portfolio with observable market state that consists of the basic open, high, low, close, and volume (OHLCV) pricing data, up to 10 technical indicators such as EMA, RSI, MACD etc., news sentiment scores generated from a fine-tuned large language model (LLM), and custom-shaped rewards.

1 Introduction

Model-free deep reinforcement learning (DRL) is a method to solve a Markov Decision Processes (MDP) using deep learning (neural networks) to approximate the policy and value functions in an environment with vast and complex state spaces, without explicitly modeling the environment's dynamics, i.e., not knowing the transition probabilities or the reward function defined in an MDP. Recall that MDP is a framework used for modeling decision making in situations where outcomes are partly random and partly under the control of a decision maker. Formally an MDP is defined by the following components: state space $\mathcal{S}$, a set of states representing different situations or configurations in which the agent or system can find itself; action space $\mathcal{A}$: a set of actions feasible to the agent. The actions that can be taken may depend on the state; transition probability function $\mathcal{P}$: often in form of a transition probability matrix $\mathcal{P}(s', s, a)$, which defines the probability of moving from state s to state s' after taking action a. This encapsulates the dynamics of the environment. The Markov property stipulates that these probabilities depend only on the current state and action, not on the sequence of events that preceded them; reward function $R(s, a, s')$: gives the immediate reward (or cost) received after transitioning from state s to state s', due to action a. This function quantifies the desirability of an outcome, allowing for policy evaluation; and a discount factor: typically denoted as $\gamma \in (0, 1)$ that models the present value of future rewards, expressing the trade-off between immediate and future rewards. The objective in a Markov Decision Process is typically to find a policy that maximizes some cumulative function of the random rewards, usually the expected discounted sum over a potentially infinite horizon:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \mid s_0 = s, \pi\right]$$

where $V^{\pi}(s)$ is the value of state s under policy π, representing the expected return starting from s and following π thereafter.

In this project, I choose to run empirical experiments based on financial market data because the compute required is much more feasible for me to handle compared to large volume of textual or imagery data, though I still have cases where I do not have enough disk space to save temporary model files generated during training (on the order of hundreds of gigabytes), which prevents me from obtaining results from even larger experiment configurations. Nonetheless, the experiment results included in later section do provide some insights that is worth further exploration in the future.

In my experiment, a simple trading task is modeled as a five-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma)$ MDP where the state space $\mathcal{S}$ encompasses the observable market state and portfolio state, the action space $\mathcal{A}$ is bounded within a pre-defined range, the transition probability function $\mathcal{P}$ is unknown and the reward function $R(s_t, a_t, s_{t+1})$ is custom-shaped based on portfolio value change. A trading agent learns a policy $\pi(s_t|a_t)$ that maximizes the discounted cumulative return over a finite trading horizon. One novel feature in this project is the leverage position and margin interesting tracking in agent's portfolio state, which has been rarely studied nor implemented before in the open-source domain to the best of my knowledge.

2 Literature Review

Applying novel models and algorithms in financial markets has never been a new idea, as shown by the renowned work like Louis Bachelier's 1900 doctoral thesis, "The Theory of Speculation" [1], which introduced the idea of using Brownian motion to model stock market prices, predating Albert Einstein's work on Brownian motion [2] and several decades before the formal development of modern financial mathematics [4, 5, 6, 7, 8]. DRL would not be an exception. The existing research on DRL in the domain of trading primarily falls into three categories: value-based (critic) algorithms that focus solely on learning the value function that determines which action to take, rather than directly learning a policy; policy-based (actor) algorithms that focus solely on learning the policy that maps states to actions, without explicitly estimating the value function; and actor-critic algorithms that use two models: the actor that estimates the policy function of the agent, mapping states to actions, and the critic that estimates the state value function or the state-action value function, providing feedback on the quality of the actions chosen by the actor.

The value-based (critic) approach [9, 15, 20], particularly the Deep Q-Network (DQN) [12], is the most extensively documented. In this approach, the state-action value function, Q, is learned to evaluate the effectiveness of a specific action within a given state. Studies using this approach generally employ a discrete action space, e.g., $\{-1, 0, 1\}$, training agents to fully commit to a position, i.e., all for selling (going short), or for all holding, or all for buying (going long), since discrete action space does not specify the size of the position. This strategy can be quite risky during periods of high market volatility, as it could lead to significant risks in the event of adverse market movements. Ideally, it would be more effective to adjust the size of positions in response to the prevailing market conditions.

The policy-based (actor) approach [10, 11], on the other hand, focuses on directly optimizing a differentiable objective function, such as profits or Sharpe ratio, with respect to the policy parameters. This method is particularly adaptable to a continuous action space, as it involves directly learning a policy. Studies [11] have shown that offline batch gradient ascent methods can effectively optimize these differentiable objective functions. These approaches use the policy gradient theorem [22] and Monte Carlo methods [3] in training. Models are updated at the end of each episode, but this often leads to slow learning. The process requires a considerable number of samples to achieve an optimal policy. This is because the method may mistakenly deem individual suboptimal actions as "good" if they contribute to overall positive rewards, resulting in a delay in correcting these actions. [24]

The actor-critic approach, such as Advantage Actor-Critic (A2C) [16], Deep Deterministic Policy Gradient (DDPG) [13], and Soft Actor-Critic (SAC) [19], with integration of indicators such as EMA, RSI, MACD, etc., have also been refined and adapted for financial trading in recent years. For recent advances in reinforcement learning in finance, I extend interested readers the following surveys: [18, 25, 29].

3 Data

Historical equity pricing data (OHLCV) and news texts are fetched from Alpaca market API and Alpaca news API. Ticker `ASML` is used for training and testing and NASDAQ-100 index (Ticker `QQQ`) is used for market benchmark. Technical indicators EMA3, EMA5, EMA10, EMA15, RSI, MACD, MACD signal, MACD historical, MFI, DX are generated from OHLCV data. HTML tags, URLs, and markdown or special characters are removed from the raw news text before feeding into a LLM for news sentiment score generation. 3 language models FinBERT, fine-tuned distilRoBERTA, and FinGPT are tested for news sentiment score generation. FinBERT is selected out of the three balancing inferencing speed and accuracy on hand-picked test examples. Since the input token length of FinBERT is capped at 512, long news text is chunked to smaller pieces using a sliding window size of 450 with stride size of 225. The final news sentiment score is obtained through majority voting. The possible news sentiment scores are $\{-1, 0, 1\}$ representing negative sentiment, neutral, and positive sentiment. News sentiment scores are left-joined to the pricing data and technical indicators on timestamps and any gap is forward-filled. In other words, the news sentiment score does not change until a new sentiment score is available. After integrating the technical indicators and news sentiment scores, the data is split 80-20 for train (training and hyperparameter tuning/cross validation) and test. And the sizes for train and test data are (508536, 16), (127135, 16).

4 Model

The observable market state consists of the pricing data, technical indicators, and news sentiment scores, all of which are normalized to within order of 10 for smoother propagation of gradients in training the neural networks. The portfolio state consists of portfolio value, cash balance, asset position, and leverage size, all scaled as well to close to order of 10. The initial cash balance of the agent is set to 1e6.

The action of the agent is obtained from a bound continuous space $[-50, 50]$ and capped to an integer value. Action value in $[-3, 3]$ are set to 0 as 3 as picked as the order size threshold; sizes smaller than 3 are deemed insignificant. In addition, a transaction cost rate of 1e-3 is applied on all orders.

Two main reward functions are tested based on portfolio value change:

- $R(s, a, s') = v' - v$ where v' and v are portfolio values at state s' and s.
- $R(s, a, s') = v' - v + \kappa_1\sqrt{t}|o| - \kappa_2 n_i - \kappa_3|h| - \kappa_4|\frac{v'}{2} - b| - \kappa_5|l|$ where t is the number of time step, o is the order size, n_i is the number of consecutive inaction greater than 30, h is the mean asset position size over the past 350 time steps [1], b is the cash balance, l is the leverage size, and κ_1 to κ_5 are hyperparameters set to 1e-6, 1e-1, 1e-2, 1e-5, and 1e-2 respectively. The idea is to encourage exploration, penalize consecutive inaction, penalize persistent large position, penalize large leverage size, and penalize the different between half of portfolio value and cash balance.

The reward discount factor is set as 0.99999990717, converting a 5% annual inflation rate to per-minute discounting rate.

A simple order and leverage mechanism at each time step is implemented as follows:

Algorithm 1 Order and leverage mechanism at each time step

```
if leverage is allowed then
    compute leverage size and leverage maintenance requirement
    if short-leveraged and insufficient portfolio value or cash balance then
        force-buy assets to cover borrowed shares; update cash balance and asset positions
        clear leverage maintenance requirements
        if negative cash balance then
            if there are positions to liquidate then
                liquidate all positions for cash, update cash balance and asset positions
                if cash balance becomes negative then
                    reset trading agent                          ▷ margin call and owe money to the broker
                end if
            else
                reset trading agent                              ▷ margin call and owe money to the broker
            end if
        end if
    end if
    if long-leveraged and insufficient portfolio value or cash balance then
        force-sell assets to cover borrowed capital; update cash balance and asset position
        clear leverage maintenance requirements
        if negative cash balance then
            if there are positions to liquidate then
                liquidate all positions for cash, update cash balance and asset positions
                if cash balance becomes negative then
                    reset trading agent                          ▷ margin call and owe money to the broker
                end if
            else
                reset trading agent                              ▷ margin call and owe money to the broker
            end if
        end if
    end if
    if order needs leverage then                                 ▷ checking for both negative and positive orders
        compute borrow size, total leverage size, and margin requirement   ▷ accounting for any existing leverage size
        if margin requirement met then                           ▷ assuming non-collateral leverage
            record leverage's timestamp, maintenance requirement, underlying asset price, borrow size, margin interest
        else
            adjust order size to feasible order size
        end if
    end if
    iteratively cover any short positions (in descending order) if order is long
    update cash balance and asset position                       ▷ (adjusted) order executed
else                                                             ▷ if leverage is not allowed
    adjust order size to feasible order size
    update cash balance and asset position                       ▷ (adjusted) order executed
end if
```

[1] 350 is hand-picked as there are $60 \cdot 6.5 = 390$ trading minutes per trading day for the equity market.

The learning algorithm chosen for the experiment is proximal policy optimization (PPO) [17], an on-policy algorithm that seeks to balance the benefits of policy gradient methods with the stability and reliability of trust region optimization with customized LSTM or attention wrapper. Training of the agent is done in a customized OpenAI gym environment [14] using Ray's RLlib [21], which supports for production-level, highly distributed RL workloads while maintaining unified and simple APIs for a large variety of industry applications. The initial learning rate is set as 5e-4 and is halved as time steps go to 5e5 and is halved again as time steps are doubled. This halving procedure stops until time steps reach 2e7. Training batch size is set as 1000, and training iteration is set from 100, then 500, and above. Note that when training iteration is set to 500, it corresponds to 500,000 time steps, which is about the same length of the training data (508536). Model parameters are tuned using Ray's Tuner and the seed is set to 2 for result reproducibility. For details, I present interested readers the actual implementation, which contains my end-to-end data processing, training, logging, inferencing, and evaluation pipelines.

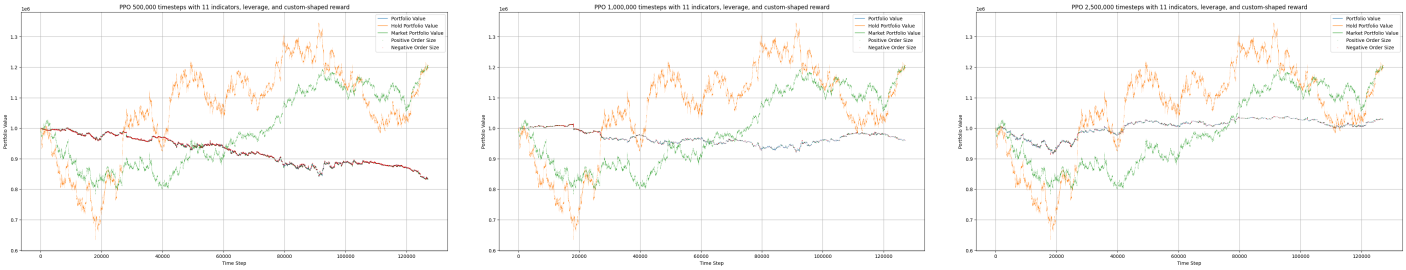
5 Results

(a) No leverage, no reward shaping, 0 indicator (complete inaction; no order made)

(b) Leverage, shaped reward, 0 indicator

(c) Leverage, shaped reward, 10 indicators, 1 sentiment score

Figure 1: Performance benchmark, with OHLCV features and more (quick training on ∼1/5 of training data)



(a) Leverage, shaped reward, 11 features, 500 training iterations
Short positions' PnL: -65,685.31
Short positions' occurrence: 7,531
No positions' PnL: +86,699.16
Long positions' PnL: -92,543.86
Long positions' occurrence: 7,822
Total PnL: -61,239.72

(b) Leverage, shaped reward, 11 features, 1000 training iterations
Short positions' PnL: -194,600.01
Short positions' occurrence: 13,065
No positions' PnL: +85,061.11
Long positions' PnL: -30,205.86
Long positions' occurrence: 16,049
Total PnL: -167,071.85

(c) Leverage, shaped reward, 11 features, 2500 training iterations
Short positions' PnL: 327.65
Short positions' occurrence: 1,727
No positions' PnL: -35,815.84
Long positions' PnL: 548.92
Long positions' occurrence: 2,106
Total PnL: -40,198.90

Figure 2: Performance benchmark, with leverage, shaped reward, and full 11 features

Figure 1 shows the effect of reward shaping and additional features when the agent is trained on the first 1/5 of the training data. As more features are added, the performance starts to pick up as Figure 1c shows. Figure 2 shows how the performance picks up when the agent is trained over time, looping through 1/5 of the entire training data, then looping roughly the entire training data, and looping the entire training data roughly 5 times [2]. As Figure 2c shows, both short and long positions begin to make positive profits. Agent's portfolio is compared against the market benchmark picked as the NASDAQ-100 index (ticker `QQQ`) and the long-only holding portfolio. Evaluation metrics used are net profit and loss (PnL), maximum drawdown and Sharpe ratio.

[2]Training iterations beyond this would require hundreds of gigabytes to save intermediate model files generated during training and a local machine may not have enough disk space, as happened to me when running experiments with more training iterations.

6 Conclusion

In conclusion, this project empirically tests the feasibility of a trading agent that is leverage-enabled using custom-shaped reward. Though not as stellar as documented in [24, 27], the performance of the model-free DRL algorithm we picked indeed increases with higher observable state dimension, larger training data volume, more efficient exploration, and proper reward shaping.

For future work, custom-shaped rewards can be improved [28] as there are many documented designs [18, 24, 23] that are much more sound than the empirically constructed ones used in this project. Equity pricing data may also be replaced by futures pricing data as futures market is known for its high liquidity and volume with nearly 24-hour trading capability, which ensure smoother price data and better market depth. The built-in leverage of futures also allows for greater capital efficiency with no short-selling restrictions, while lower transaction costs benefit high-frequency transactions. While this project extensively explored available options of the PPO algorithm in Ray's framework, there are many other popular DRL algorithms, such as the Asynchronous Advantage Actor-Critic (A3C) algorithm [16], worth to test. A3C utilizes multiple workers (agents) that explore the environment independently on multiple threads, which helps in exploring the state space more diversely. While A3C can be more efficient in a distributed setting with multiple CPU cores, sometimes it can lead to unstable training or convergence issues, especially in environments with a lot of variance in returns. Nonetheless, it is worth to try out other DRL algorithms, such as the recent breakthrough with a framework called `MEX` [30], to examine the robustness of agent's performance. Last but not least, financial market data can also be leveraged to learn many other tasks through DRL, e.g., market making [26]. DRL is a vibrant yet vast domain, with many directions for interested readers to explore, such as Multi-Agent Reinforcement Learning (MARL), Partially Observable Markov Decision Processes (POMDPs), Offline Reinforcement Learning, etc.

References

[1] Louis Bachelier. "Théorie de la spéculation". In: *Annales scientifiques de l'École normale supérieure*. Vol. 17. 1900, pp. 21–86.

[2] Albert Einstein. "On the Motion of Small Particles Suspended in Liquids at Rest Required by the Molecular-Kinetic Theory of Heat". In: *Annalen der physik* 17.549-560 (1905), p. 208.

[3] Nicholas Metropolis and Stanislaw Ulam. "The Monte Carlo Method". In: *Journal of the American statistical association* 44.247 (1949), pp. 335–341.

[4] Harry M Markowits. "Portfolio Selection". In: *Journal of finance* 7.1 (1952), pp. 71–91.

[5] William F Sharpe. "Capital asset prices: A theory of market equilibrium under conditions of risk". In: *The journal of finance* 19.3 (1964), pp. 425–442.

[6] Eugene F Fama. "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The journal of Finance* 25.2 (1970), pp. 383–417.

[7] Fischer Black and Myron Scholes. "The Pricing of Options and Corporate Liabilities". In: *Journal of political economy* 81.3 (1973), pp. 637–654.

[8] Robert J. Shiller. "The Use of Volatility Measures in Assessing Market Efficiency". In: *The Journal of Finance* 36.2 (1981), pp. 291–304.

[9] Ralph Neuneier. "Optimal Asset Allocation using Adaptive Dynamic Programming". In: *Advances in neural information processing systems* 8 (1995).

[10] John Moody et al. "Performance Functions and Reinforcement Learning for Trading Systems and Portfolios". In: *Journal of forecasting* 17.5-6 (1998), pp. 441–470.

[11] John Moody and Matthew Saffell. "Learning to Trade via Direct Reinforcement". In: *IEEE transactions on neural Networks* 12.4 (2001), pp. 875–889.

[12] Volodymyr Mnih et al. "Playing Atari with Deep Reinforcement Learning". In: *arXiv preprint arXiv:1312.5602* (2013).

[13] Timothy P Lillicrap et al. "Continuous Control with Deep Reinforcement Learning". In: *arXiv preprint arXiv:1509.02971* (2015).

[14] Greg Brockman et al. "OpenAI Gym". In: *arXiv preprint arXiv:1606.01540* (2016).

[15] Olivier Jin and Hamza El-Saawy. "Portfolio Management using Reinforcement Learning". In: *Stanford University* (2016).

[16] Volodymyr Mnih et al. "Asynchronous Methods for Deep Reinforcement Learning". In: *International conference on machine learning*. PMLR. 2016, pp. 1928–1937.

[17] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[18] Thomas G Fischer. *Reinforcement learning in financial markets-a survey*. Tech. rep. FAU Discussion Papers in Economics, 2018.

[19] Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *International conference on machine learning*. PMLR. 2018, pp. 1861–1870.

[20] Chien Yi Huang. "Financial Trading as a Game: A Deep Reinforcement Learning Approach". In: *arXiv preprint arXiv:1807.02787* (2018).

[21] Eric Liang et al. "RLlib: Abstractions for distributed reinforcement learning". In: *International conference on machine learning*. PMLR. 2018, pp. 3053–3062.

[22] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT press, 2018.

[23] Yang Liu et al. "Adaptive Quantitative Trading: An Imitative Deep Reinforcement Learning Approach". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 02. 2020, pp. 2128–2135.

[24] Zihao Zhang, Stefan Zohren, and Roberts Stephen. "Deep Reinforcement Learning for Trading". In: *The Journal of Financial Data Science* (2020).

[25] Adrian Millea. "Deep Reinforcement Learning for Trading - A Critical Survey". In: *Data* 6.11 (2021), p. 119.

[26] Rama Cont and Wei Xiong. "Dynamics of market making algorithms in dealer markets: Learning and tacit collusion". In: *Mathematical Finance* (2022).

[27] Xiao-Yang Liu et al. "FinRL-Meta: Market Environments and Benchmarks for Data-Driven Financial Reinforcement Learning". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 1835–1849.

[28] Serena Booth et al. "The Perils of Trial-and-Error Reward Design: Misdesign through Overfitting and Invalid Task Specifications". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 5. 2023, pp. 5920–5929.

[29] Ben Hambly, Renyuan Xu, and Huining Yang. "Recent Advances in Reinforcement Learning in Finance". In: *Mathematical Finance* 33.3 (2023), pp. 437–503.

[30] Zhihan Liu et al. "Maximize to Explore: One Objective Function Fusing Estimation, Planning, and Exploration". In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023.